

## APPENDIX A

### DATASET AND EVALUATION DETAILS

The following appendices give the dataset, training, and ablation details that complement Sec. IV of the main paper. Full rollout videos and additional qualitative comparisons are hosted on the project website<sup>[1]</sup>.

#### A. UR5 World Model Training Dataset

We train SC3-Eval on the real-world UR5 dataset introduced in Sec. IV-A, which spans 381 hours of robot trajectories collected in a single physical scene with 12 object categories. Each trajectory provides synchronized streams from three camera views, two third-person cameras and one wrist-mounted camera, recorded at 20 Hz with native resolution  $480 \times 640$ . We additionally subsample each stream to 10 Hz to form the multi-FPS training mix described in App. B-D.0.b.

The 12 object categories cover yellow cup, orange cup, blue bowl, white bowl, orange plate, chopstick, spoon, chip bag, plastic lid, plastic container, foil tray, and bottle.

Each action is recorded as a delta end-effector pose (delta-EE) and represented as a 7-dimensional vector with three components for translation, three for axis-angle rotation, and one for gripper width. We normalize each dimension to unit variance using statistics computed over the full training corpus.

#### B. UR5 Policy Evaluation Setup

Our real-world evaluation covers seven  $\pi_{0.5}$  [26] policy checkpoints. For each checkpoint we collect 36 matched initial conditions, each rolled out both on the physical robot and inside our world-model evaluator from the same starting state for direct comparison. All rollouts, whether physical or closed-loop within the world model, run for at most 20 seconds, so that every mode shares the same temporal budget.

Tab. III reports the per-criterion real-world success rates measured in the physical-robot rollouts for all seven UR5 policy checkpoints. Each checkpoint predicts 25 actions at 10 Hz, and only the first 16 actions are executed under a receding-horizon strategy before the next replan.

**TABLE III: Real-world performance of the evaluated UR5 policies.** Each row reports the real-world success rate per success criterion.

| Policy<br>Checkpoint ID | Real-world success rate |                   |                   |
|-------------------------|-------------------------|-------------------|-------------------|
|                         | Language<br>following   | Object<br>lifting | Object<br>placing |
| 1                       | 0.946                   | 0.610             | 0.126             |
| 2                       | 0.876                   | 0.688             | 0.306             |
| 3                       | 0.959                   | 0.720             | 0.350             |
| 4                       | 0.446                   | 0.154             | 0.008             |
| 5                       | 0.745                   | 0.444             | 0.286             |
| 6                       | 0.675                   | 0.105             | 0.057             |
| 7                       | 0.926                   | 0.785             | 0.449             |

<sup>[1]</sup><https://miscsubmission.github.io/sc3-eval/>

#### C. UR5 Scoring and Annotation

This subsection details the criteria used to score each rollout and the protocol followed by human annotators.

a) *Success criteria*: Each rollout is scored on three binary criteria described in Sec. IV-A of the main paper, *language following*, *object lifting*, and *object placing*, with each criterion scored separately for every object involved in the rollout. For each rollout, three annotators score the rollout on each criterion, and the success rate for each criterion is the mean of the three annotator scores. Trajectories from the two sources (real-world and closed-loop) are rendered at a uniform  $480 \times 640$  resolution and presented to annotators in randomized order. Source labels are stripped from filenames and metadata, and annotators are not told the proportion of trajectories drawn from each source. The 95% confidence intervals reported in Tab. I are estimated using 1,000 bootstrap resamples of rollouts.

b) *Failure-mode taxonomy*: For each *failed* trajectory, annotators additionally assign exactly one of the three failure modes used in Sec. IV-C of the main paper, *language-following failure*, *object-lifting failure*, or *object-placing failure*. The taxonomy mirrors the three success criteria, and each failed trajectory is assigned to the first criterion it violates (in the order language  $\rightarrow$  lifting  $\rightarrow$  placing).

#### D. YAM World Model Training Dataset

For YAM, we train SC3-Eval on the MolmoAct2-BimanualYAM dataset [9], comprising around 720 hours of robot demonstrations. The early-termination threshold is selected using a held-out subset comprising 0.5% of the dataset’s trajectories.

Similar to UR5, YAM setup also consists of three camera views: left wrist, right wrist, and egocentric. Each view has an image resolution of  $640 \times 360$ . We temporally downsample the data from 30 FPS to 15 FPS for training. YAM actions use the same delta end-effector pose (delta-EE) representation as UR5 (App. A-A), extended to two arms. Each arm contributes a 7-dimensional vector with three components for translation, three for axis-angle rotation, and one for gripper width, giving a 14-dimensional bimanual action vector. We use the same per-dimension unit-variance normalization procedure, with statistics computed over the YAM training dataset.

#### E. YAM Policy Evaluation Setup

We evaluate four policies, namely MolmoAct2-YAM [9], GR00T-N1.7 [27],  $\pi_{0.5}$  [26], and OpenVLA-OFT [28], on tasks involving picking up a sponge, a pen, a gray cup, and a blue cup.

We use the publicly available checkpoints of MolmoAct2-YAM<sup>[2]</sup>, GR00T-N1.7<sup>[3]</sup> and  $\pi_{0.5}$ <sup>[4]</sup> from Hugging Face.

<sup>[2]</sup>[https://huggingface.co/allenai/](https://huggingface.co/allenai/MolmoAct2-BimanualYAM)

MolmoAct2-BimanualYAM

<sup>[3]</sup>[https://huggingface.co/Jiafei1224/](https://huggingface.co/Jiafei1224/molmoact2-yam-gr00t1.7)

molmoact2-yam-gr00t1.7

<sup>[4]</sup>[https://huggingface.co/Jiafei1224/](https://huggingface.co/Jiafei1224/molmoact2-yam-pi05)

molmoact2-yam-pi05

TABLE IV: **Real-world performance of the evaluated YAM policies.** Each entry is the mean success rate across the four object-picking tasks in the standard evaluation setting.

| Policy            | Real-world success rate |                |
|-------------------|-------------------------|----------------|
|                   | Language following      | Object lifting |
| MolmoAct2-YAM [9] | 0.425                   | 0.250          |
| GR00T-N1.7 [27]   | 0.738                   | 0.438          |
| $\pi_{0.5}$ [26]  | 0.675                   | 0.400          |
| OpenVLA-OFT [28]  | 0.325                   | 0.150          |

For OpenVLA-OFT, we perform fine tuning on the MolmoAct2-BimanualYAM dataset for 20,000 training steps using the official implementation<sup>5</sup>.

Each policy is evaluated with 20 trajectories per object, yielding 80 trajectories across the four tasks. Following Sec. IV-A, rollouts on the physical robot and those generated in closed loop by the world model start from matched initial conditions and run for at most 8 seconds. Tab. IV reports the success rates on the physical robot averaged across the four tasks.

*a) Environmental distribution shifts:* We evaluate the combinations of policies and tasks shown in Fig. 8 under two shifts. For the object layout shift, we move the pen from the gray cup to the blue cup and evaluate lifting the blue cup with OpenVLA-OFT and MolmoAct2-YAM. For the object color shift, we replace the blue cup with a yellow cup and evaluate language following on the task of picking up the sponge with MolmoAct2-YAM and GR00T-N1.7. Original and modified conditions are paired by initial scene configuration, with language prompts and all other conditions held fixed. For each evaluated policy, we use 20 trajectories per object under each condition.

#### F. YAM Scoring and Annotation

YAM follows the same blinded human-annotation and success-rate aggregation protocol as UR5 (App. A-C), but uses only two binary success criteria: *language following* and *object lifting*. Similar to UR5, the confidence intervals are estimated using the same rollout-level bootstrap procedure.

### APPENDIX B TRAINING DETAILS

In this section, we describe the training and inference configurations for the UR5 and YAM experiments on SC3-Eval and baselines implementations.

#### A. Network Architecture

We initialize SC3-Eval from the pre-trained weights of Cosmos3-Nano [11] and adopt its native video-action modeling architecture.

Raw delta-EE actions are normalized per dimension as described in App. A-A and mapped into action tokens through linear input projections. These tokens are processed jointly

with video tokens by the shared transformer backbone, while linear output projections map predicted action tokens back to the action space. The multi-view forward dynamics, inverse dynamics, and cross-view inpainting training modes are described in Sec. III-B.

#### B. Hardware and Compute

We train SC3-Eval on 32 NVIDIA GB200 GPUs (8 nodes with 4 GPUs per node) for approximately 2.2 wall-clock days. Inference for closed-loop evaluation runs at 2.3 seconds per 24-frame chunk on a single GB200 GPU for UR5 setup and 1.07 seconds per 16-frame chunk for YAM setup.

#### C. Optimization

*a) Optimizer and schedule:* We use the AdamW optimizer with learning rate 0.0001, weight decay 0.05,  $\beta_1 = 0.9$ ,  $\beta_2 = 0.99$ , and a 2,000-step linear warmup followed by cosine decay. The effective batch size is 512 trajectories per step (16 per-GPU minibatch across 8 nodes with 4 GB200 GPUs per node), and we train for 24,000 total steps with the rectified-flow loss inherited from Cosmos3.

*b) Mode mixture:* Following the training procedure in Sec. III-B, we use the following mode-sampling probabilities. For UR5, we set  $p_{\text{FD}} = 0.8$  for multi-view forward dynamics,  $p_{\text{CVI}} = 0.1$  for cross-view inpainting, and  $p_{\text{ID}} = 0.1$  for inverse dynamics. For YAM, we set  $p_{\text{FD}} = 0.9$ ,  $p_{\text{CVI}} = 0.05$ , and  $p_{\text{ID}} = 0.05$ .

#### D. Training Augmentations

*a) Pseudo-action augmentation:* We use pseudo-action augmentation to expose the world model to both commanded actions and actions derived from observed robot motion. For each training sample, with probability 0.5, we replace the policy command with a pseudo-action computed from consecutive real-world end-effector poses. Mixing these two action sources broadens the action distribution encountered during training.

*b) Multi-FPS training:* We use multi-FPS training to expose the world model to both fine-grained motion and longer temporal context within a fixed frame budget. For UR5, we jointly train at 10 and 20 FPS by sampling the temporal stride uniformly per minibatch. With 24-step training chunks, the 20 FPS stream covers 1.2 seconds and provides fine-grained inter-frame information for short-range motion and contact, while the 10 FPS stream covers 2.4 seconds and captures movement over a longer temporal span. Since the transformer does not directly observe the frame rate through its positional encoding, multi-FPS sampling serves as data augmentation that broadens the temporal statistics represented by each chunk. For YAM, we train at 15 FPS only, which corresponds to an effective action horizon of 1.07 seconds for the 16-step chunks used during training.

#### E. Inference Procedure

*a) Prediction-execution horizon decoupling:* Following the notation of Sec. III-C of the main paper, we decouple the world model’s prediction horizon from the policy’s execution

<sup>5</sup><https://github.com/moojink/openvla-oft>

TABLE V: **Open-loop rollout PSNR per method, ablation variant, and camera view.** Results are reported separately for the three camera views of UR5 and YAM. PSNR is averaged over the first 50 generated frames of each rollout. Higher is better.

| Method                    | UR5 PSNR $\uparrow$ |                   |              | YAM PSNR $\uparrow$ |                |              |
|---------------------------|---------------------|-------------------|--------------|---------------------|----------------|--------------|
|                           | Third-person<br>1   | Third-person<br>2 | Wrist        | Left<br>wrist       | Right<br>wrist | Egocentric   |
| Ctrl-World [6]            | 19.90               | 19.92             | 19.01        | 19.86               | 19.79          | 20.51        |
| IRASim [3]                | 19.44               | 19.47             | 18.60        | 19.01               | 19.04          | 20.00        |
| Cosmos-Predict 2.5 [10]   | 19.86               | 19.88             | 19.07        | 19.82               | 19.80          | 20.04        |
| Cosmos3-FD [11]           | 20.41               | 20.73             | 19.92        | 20.37               | 20.45          | 21.19        |
| w/o inverse dynamics      | 21.03               | 21.05             | 19.43        | 19.93               | 19.91          | 21.56        |
| w/o cross-view inpainting | 21.33               | 21.35             | 19.71        | 19.71               | 19.69          | 21.19        |
| w/o horizon decoupling    | 20.96               | 20.98             | 19.37        | 19.99               | 19.97          | 21.89        |
| <b>SC3-Eval (Ours)</b>    | <b>21.46</b>        | <b>21.49</b>      | <b>20.64</b> | <b>21.39</b>        | <b>21.37</b>   | <b>22.10</b> |

horizon. For UR5, the world model is trained at a prediction horizon of  $l' = 24$  frames, of which only the first  $l = 16$  frames are retained at inference, matching the policy’s 16-step receding-horizon execution. Training directly at  $l = 16$  frames degrades generation quality, since this horizon is far shorter than the 400-frame video clips on which Cosmos3 was pre-trained. At each closed-loop step the policy proposes a 25-action chunk, of which we feed the first  $l' = 24$  actions to the world model. The world model generates  $l'$  future frames in each of the three camera views, and we keep the first  $l$  frames per view as the observation context for the next chunk. For YAM, SC3-Eval uses a world-model prediction horizon of  $l' = 16$  frames and an execution horizon of  $l = 8$  steps.

*b) Early-termination threshold:* For the uncertainty-driven early-termination criterion used in Sec. IV-E, we tune the UR5 per-chunk threshold to  $\tau = 0.02$  on a held-out subset of 30 trajectories. The UR5 threshold-sensitivity analysis is provided in App. C-C. For YAM, we enable uncertainty-driven early termination with threshold  $\tau = 0.45$ , selected using the held-out subset described in App. A-D. Following the main-paper protocol, terminated rollouts remain included in the success-rate computation, and annotators score each criterion using only the retained trajectory prefix.

#### F. Baseline Implementation Details

For a fair comparison in Sec. IV-A, we fine-tune all four baselines on the same dataset used to train SC3-Eval for each platform. We follow the official post-training procedure for Ctrl-World<sup>6</sup> and the official training procedure for IRASim<sup>7</sup>. For Cosmos-Predict 2.5, we use the action-conditioned post-training procedure provided in its official repository<sup>8</sup>. We obtain Cosmos3-FD by fine-tuning Cosmos3-Nano using only the forward dynamics objective, following the official training instructions<sup>9</sup>. Each baseline uses the same training

configuration across UR5 and YAM.

### APPENDIX C ADDITIONAL ABLATION STUDY

This section complements the closed-loop ablations in Tab. II with two additional analyses. We examine how the training modes and prediction-execution horizon decoupling affect open-loop visual quality on UR5 and YAM (Tab. V). We also analyze how the early-termination threshold affects closed-loop policy evaluation across all seven UR5 checkpoints.

#### A. Training Modes

SC3-Eval introduces two grounding training modes alongside multi-view forward dynamics (Sec. III-B), inverse dynamics and cross-view inpainting. We ablate each in turn below.

*a) Inverse dynamics:* As shown in Tab. V, removing inverse dynamics supervision reduces open-loop PSNR across all camera views on both platforms. Fig. 9 provides a qualitative comparison on a bottle pick-and-place trajectory. The model trained without inverse dynamics initially follows the ground-truth motion but begins to diverge after the first second, whereas the model trained with inverse dynamics continues to track the motion through the end of the rollout. Together, these results suggest that joint forward-inverse training helps reduce drift during autoregressive generation.

*b) Cross-view inpainting:* As shown in Tab. V, cross-view inpainting improves open-loop PSNR across all camera views on both platforms, with larger gains in the wrist views. This suggests that cross-view supervision is particularly useful for wrist-mounted cameras, whose viewpoints change with arm motion. Fig. 10 illustrates this benefit through a wrist-view scene re-entry example on UR5. After the wrist camera turns away from the workspace and returns, the model trained without cross-view inpainting produces inconsistent wrist-view content, whereas the model trained with cross-view inpainting recovers the correct workspace state.

<sup>6</sup><https://github.com/Robert-gyj/Ctrl-World>

<sup>7</sup><https://github.com/bytedance/IRASim>

<sup>8</sup>[https://github.com/nvidia-cosmos/cosmos-predict2.5/blob/main/docs/post-training\\_video2world\\_action.md](https://github.com/nvidia-cosmos/cosmos-predict2.5/blob/main/docs/post-training_video2world_action.md)

<sup>9</sup>[https://github.com/NVIDIA/cosmos-framework/blob/main/docs/action\\_fd\\_droid\\_posttrain.md](https://github.com/NVIDIA/cosmos-framework/blob/main/docs/action_fd_droid_posttrain.md)

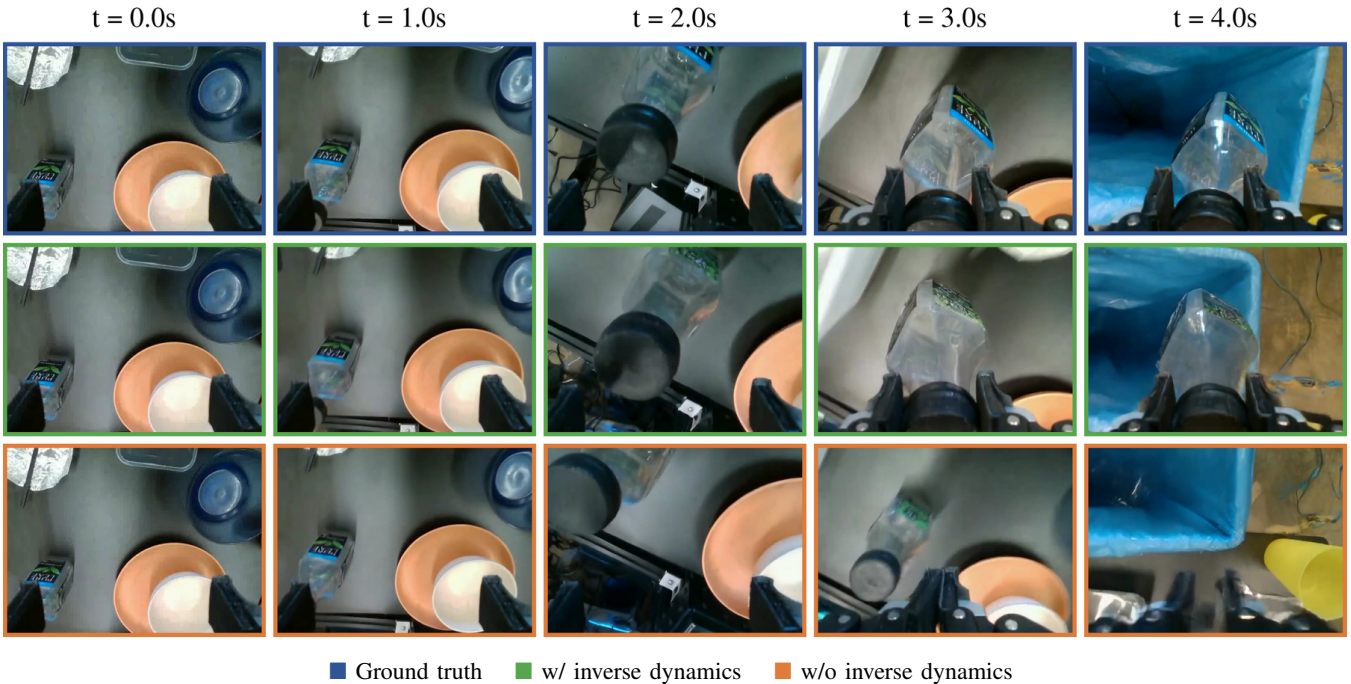


Fig. 9: **Qualitative effect of inverse dynamics grounding on open-loop rollouts.** Each row shows the same open-loop rollout under a different training configuration, sampled at five timesteps from a single bussing trajectory. The variant trained without inverse dynamics matches the ground truth for the first second, then drifts onto a different scene and never recovers, while the variant with inverse dynamics tracks the ground-truth bottle pick-and-place motion through the end of the rollout.

TABLE VI: **Uncertainty-driven early termination ablation on UR5.** Pearson  $r$  and MMRV are computed separately for each success criterion across all seven checkpoints, then averaged over the three criteria. Bold indicates the best result among the evaluated settings.

| Threshold $\tau$          | Termination rate | Mean Pearson $r \uparrow$ | Mean MMRV $\downarrow$ |
|---------------------------|------------------|---------------------------|------------------------|
| $\infty$ (no termination) | 0%               | 0.894                     | 0.080                  |
| $\tau_1 = 0.05$           | 2.21%            | 0.899                     | 0.075                  |
| $\tau_2 = 0.02$           | 4.88%            | <b>0.948</b>              | <b>0.036</b>           |
| $\tau_3 = 0.01$           | 10.22%           | 0.901                     | 0.128                  |

Increasing the threshold to  $\tau = 0.05$  yields only small improvements over no termination. Lowering it to  $\tau = 0.01$  increases the termination rate but worsens both metrics relative to the adopted setting. This lower threshold even raises MMRV above the no-termination baseline, indicating that more frequent termination can impair policy ranking. These results suggest that  $\tau = 0.02$  strikes a favorable balance in our setting.

Fig. 11 illustrates the criterion on an example rollout, where early termination is triggered when the per-chunk consistency error first exceeds the threshold.

### B. Prediction-Execution Horizon Decoupling

The prediction-execution horizon decoupling result in Tab. III of the main paper carries over to the open-loop mode, with the w/o horizon decoupling row of Tab. V showing a PSNR drop in every camera view on both platforms. Setting  $l' = l$  (no decoupling) shortens the per-chunk video horizon below the world model’s pre-training prior, and degrades both closed-loop policy-level correlation and open-loop visual quality.

### C. Uncertainty-Driven Early Termination

We examine how the early-termination threshold affects closed-loop policy evaluation on UR5. As shown in Tab. VI, the adopted threshold  $\tau = 0.02$  achieves the highest mean Pearson correlation and lowest mean MMRV among the evaluated settings, improving both metrics over no termination.

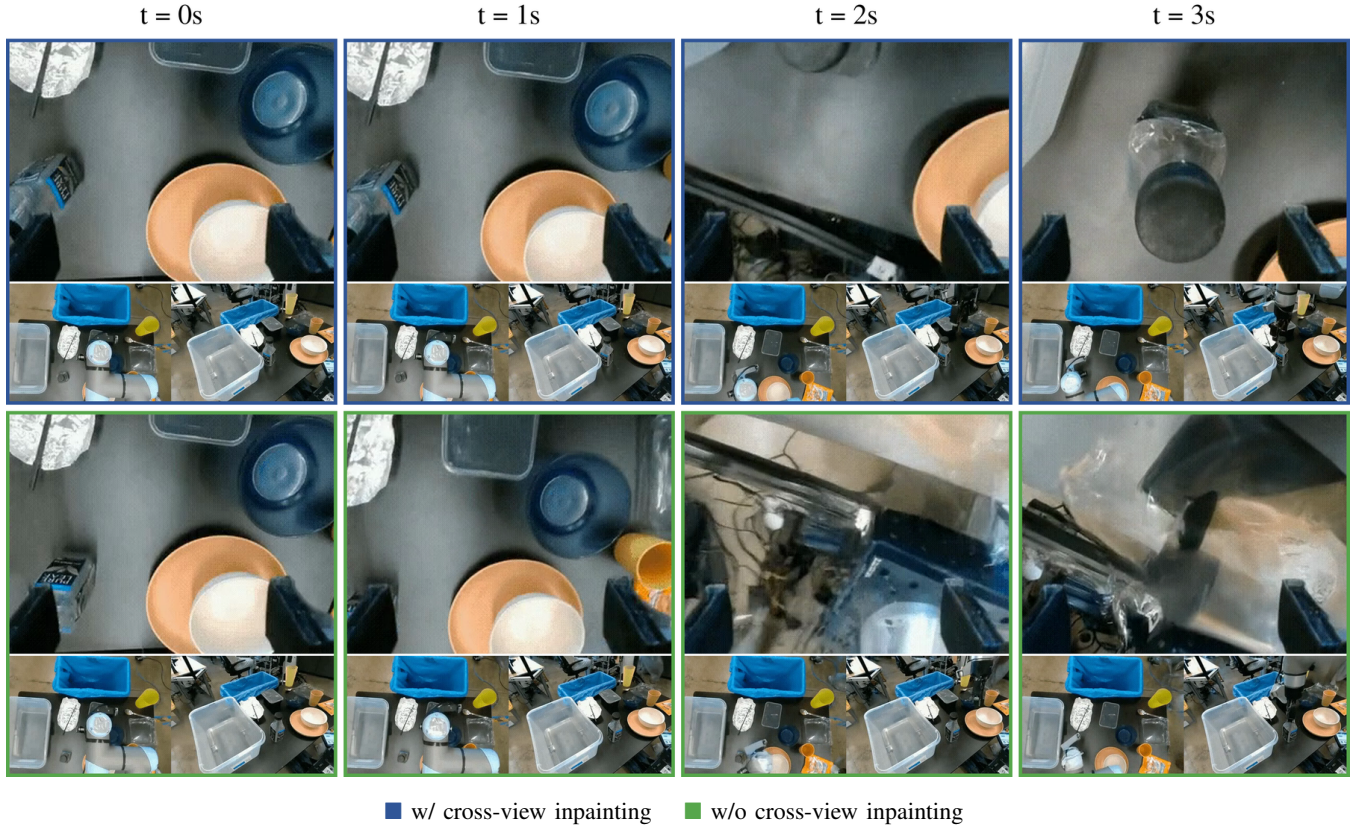


Fig. 10: **Qualitative effect of cross-view inpainting on wrist-view scene re-entry.** Each row shows the same open-loop rollout under a different training configuration, sampled at four timesteps spanning a re-entry segment in which the wrist camera turns away from the workspace and then returns. Each panel shows a zoomed wrist view on top and the two third-person views below. The variant trained without cross-view inpainting hallucinates the wrist content and never resolves once the workspace becomes visible again, while the variant trained with cross-view inpainting recovers the correct workspace state under occlusion.

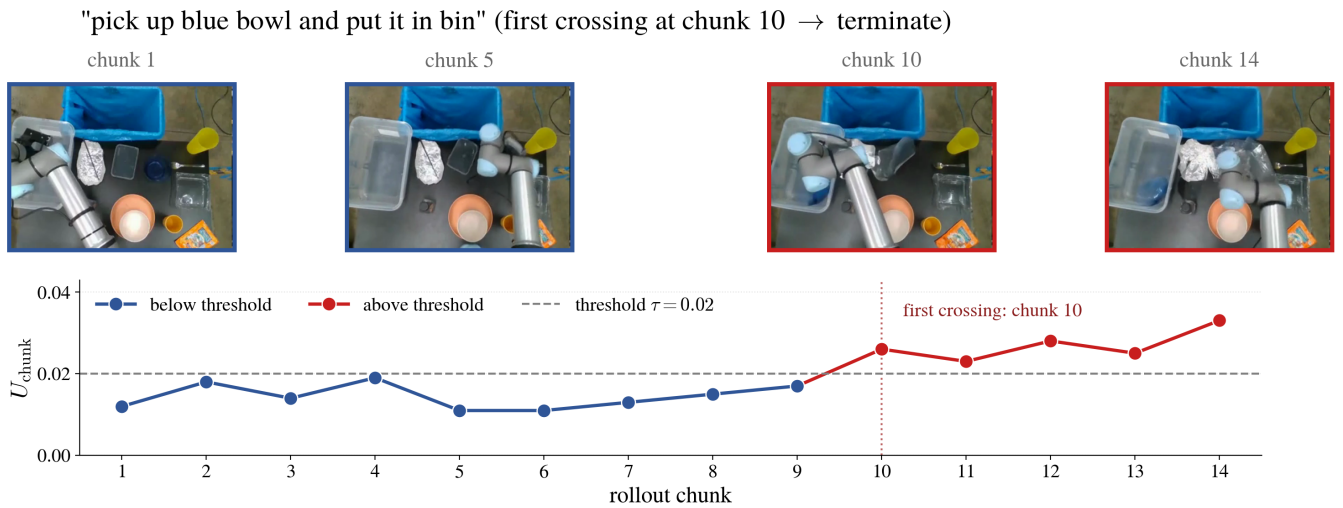


Fig. 11: **Uncertainty-driven early termination on an example rollout.** Top, four frames sampled from the imagined rollout. Bottom, the per-chunk consistency error  $U_{\text{chunk}}$  over the 14 rollout chunks. The rollout is terminated at the first chunk whose  $U_{\text{chunk}}$  exceeds the threshold  $\tau = 0.02$  (chunk 10 here), before the visible drift compounds.